

Corrigé du TD

Les algorithmes de tri

Complexité • Correction • Terminaison

CPGE — filières MP / PSI | Programme officiel marocain 2023

Partie I — Les tris quadratiques

Exercice 1 — Tri par sélection : comptage exact et invariant

Solution. 1. Pour chaque $i \in \{0, \dots, n-2\}$, la boucle interne parcourt $j \in \{i+1, \dots, n-1\}$, soit $n-1-i$ comparaisons. Au total

$$\sum_{i=0}^{n-2} (n-1-i) = \sum_{p=1}^{n-1} p = \frac{n(n-1)}{2}.$$

Ce nombre ne dépend *pas* du contenu de t : les deux boucles sont des **for** de bornes fixées.

2. Un échange par tour de la boucle externe, soit exactement $n-1$ échanges (y compris les échanges « à vide » lorsque $\text{imin} = i$). Meilleur cas comme pire cas : $\Theta(n^2)$. Le tri par sélection est le seul des tris étudiés dont la complexité ne dépend pas de l'entrée.

3. Terminaison. Les deux boucles sont des **for** sur des intervalles finis : elles terminent. (Formellement, le variant de la boucle interne est $n-j$, celui de la boucle externe $n-1-i$.)

4. Invariant $\mathcal{I}_i$: « $t[0..i-1]$ est trié par ordre croissant et contient les i plus petits éléments du tableau initial (multiensemble) ».

Initialisation. Pour $i = 0$, $t[0..-1]$ est vide : $\mathcal{I}_0$ est vraie.

Hérédité. Supposons $\mathcal{I}_i$ vraie. La boucle interne calcule $\text{imin} = \arg \min_{i \leq j \leq n-1} t[j]$ (invariant interne immédiat : « $t[\text{imin}] = \min t[i..j-1]$ »). Comme $t[0..i-1]$ contient les i plus petits éléments, $t[\text{imin}]$ est le $(i+1)$ -ième plus petit élément du tableau initial, et il est $\geq$ tous les éléments de $t[0..i-1]$. Après l'échange, $t[0..i]$ est donc trié et contient les $i+1$ plus petits éléments : $\mathcal{I}_{i+1}$ est vraie.

Conclusion. À la sortie, $i = n-1$ et $\mathcal{I}_{n-1}$ dit que $t[0..n-2]$ est trié et contient les $n-1$ plus petits éléments ; le dernier élément restant est donc le maximum et t est trié. De plus seules des transpositions ont été effectuées : le résultat est bien une permutation de l'entrée.

5. Précisément pour la raison ci-dessus : quand les $n-1$ plus petits éléments sont placés, le dernier l'est automatiquement. Un tour supplémentaire serait correct mais inutile.

Exercice 2 — Tri par insertion : le rôle des inversions

Solution. 1. Sur $t = [5, 2, 4, 6, 1, 3]$:

i	<code>c1e</code>	tableau après l'itération
1	2	[2, 5 4, 6, 1, 3]
2	4	[2, 4, 5 6, 1, 3]
3	6	[2, 4, 5, 6 1, 3]
4	1	[1, 2, 4, 5, 6 3]
5	3	[1, 2, 3, 4, 5, 6]

(la barre sépare la partie déjà triée du reste.)

2. Si t est trié, le test $t[j] > \text{c1e}$ est faux dès la première évaluation : une comparaison par valeur de i , soit $n - 1$ comparaisons et 0 décalage. Complexité $\Theta(n)$.

3. Si t est trié en ordre strictement décroissant, pour chaque i la boucle **while** parcourt tout $j = i - 1, \dots, 0$ puis s'arrête sur $j \geq 0$ faux : i comparaisons et i décalages. Total :

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \text{ comparaisons et autant de décalages, } \quad \Theta(n^2).$$

4. Considérons un décalage $t[j+1] = t[j]$: il fait passer l'élément $t[j]$ (qui vérifie $t[j] > \text{c1e}$) à droite de `c1e`. Le couple $(t[j], \text{c1e})$ était une inversion : il cesse de l'être. Aucun autre couple ne voit son ordre relatif modifié par ce décalage (les éléments de $t[0..j-1]$ et ceux de $t[i+1..]$ ne bougent pas, et les éléments déplacés le sont tous d'une case vers la droite, donc dans le même ordre entre eux). Chaque décalage supprime donc *exactement* une inversion et n'en crée aucune. Comme le tableau final n'a plus d'inversion, le nombre total de décalages vaut $I(t)$.

5. Le coût total est : $n - 1$ tours de boucle externe à $\Theta(1)$ chacun, plus $I(t)$ décalages, plus au plus une comparaison « d'arrêt » par tour. D'où

$$\Theta(n + I(t)).$$

Le tri est donc *adaptatif* : sur un tableau presque trié, $I(t)$ est petit et le coût est quasi linéaire. Si chaque élément est à distance $\leq k$ de sa position finale, alors $I(t) \leq nk$ et la complexité est $\Theta(nk)$: linéaire à k fixé. Dans le pire des cas $I(t) = \binom{n}{2}$ et l'on retrouve $\Theta(n^2)$.

Exercice 3 — Tri par insertion : preuves complètes

Solution. 1. *Variant* : $V = j + 1$. C'est un entier, positif ou nul tant que la condition $j \geq 0$ est vraie, et chaque tour exécute $j -= 1$, donc V décroît strictement de 1. La boucle termine (en au plus i tours).

Quant à l'indice : le **and** de Python est *paresseux* , il n'évalue $t[j] > \text{c1e}$ que si $j \geq 0$ est vrai. Sans cela, $j = -1$ donnerait en Python $t[-1]$ — le *dernier* élément du tableau — et l'algorithme serait faux sans lever d'erreur. C'est un piège classique : l'ordre des deux conditions est essentiel.

2. *Invariant $\mathcal{J}$* : « $t[0..j]$ est inchangé et trié ; $t[j+2..i]$ contient, dans le même ordre, les éléments initialement en $t[j+1..i-1]$, et ils sont tous $> \text{c1e}$; la valeur `c1e` est sauvegardée ».

Initialisation. Avant le premier tour, $j = i - 1$: $t[j+2..i] = t[i+1..i]$ est vide et $t[0..i-1]$ est trié par l'invariant externe. ✓

Hérédité. Si le test est vrai, $t[j] > \text{c1e}$; après $t[j+1] = t[j]$ puis $j -= 1$, la tranche $t[j'+2..i]$ (avec $j' = j - 1$) contient exactement l'ancienne tranche augmentée de $t[j]$, en tête, tous $> \text{c1e}$, et $t[0..j']$ est inchangé et trié. ✓

Sortie. La boucle s'arrête soit parce que $j = -1$, soit parce que $t[j] \leq \text{c1e}$. Dans les deux cas tous les éléments de $t[0..j]$ sont $\leq \text{c1e}$ (le tableau $t[0..i-1]$ était trié) et tous ceux de $t[j+2..i]$ sont $> \text{c1e}$: l'affectation $t[j+1] = \text{c1e}$ place `c1e` à sa place et rend $t[0..i]$ trié.

- 3.** *Invariant $\mathcal{I}_i$* : « $t[0..i-1]$ est trié et est une permutation des i premiers éléments initiaux ». Initialisation : $i = 1$, un seul élément. Hérédité : c'est exactement la conclusion du point **2**. Sortie : $i = n$, donc $t[0..n-1]$ est trié et est une permutation de t . ✓
- 4.** Les seules variables auxiliaires sont `cle`, `i` et `j` : $\Theta(1)$ mémoire supplémentaire, indépendamment de n . Le tri par insertion est donc **en place**.

Exercice 4 — Tri à bulles et arrêt anticipé

Solution. 1. *Invariant interne* : « après le tour d'indice j , $t[j+1] = \max t[0..j+1]$ ». Initialisation ($j = 0$) : le test place le plus grand de $t[0], t[1]$ en $t[1]$. Hérédité : si $t[j] = \max t[0..j]$, alors après le test/échange à l'indice j , $t[j+1] = \max(t[j], t[j+1]) = \max t[0..j+1]$. ✓

Invariant externe $\mathcal{I}_i$: « après l'itération i , la tranche $t[n-1-i..n-1]$ est triée et contient les $i+1$ plus grands éléments ». En effet la boucle interne, parcourue jusqu'à $j = n-2-i$, amène le maximum de $t[0..n-1-i]$ en position $n-1-i$, lequel est $\leq$ tous les éléments déjà placés à droite.

2. Variant interne : $(n-1-i) - j$; variant externe : $(n-1) - i$.

3. Pire des cas (aucun arrêt anticipé) : $\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$ comparaisons, soit $\Theta(n^2)$.

4. Si t est déjà trié, aucun échange n'a lieu au premier passage : `échange` reste `False` et l'algorithme renvoie après $n-1$ comparaisons. Complexité $\Theta(n)$; la famille est celle des tableaux déjà triés.

5. Un échange n'a lieu que si $t[j] > t[j+1]$, c'est-à-dire sur une inversion de deux éléments *adjacents* ; l'échange la supprime et ne modifie l'ordre relatif d'aucun autre couple. Donc le nombre d'échanges vaut exactement $I(t)$ — même comptage que pour les décalages du tri par insertion.

Comparaison. Les deux tris font le même nombre de « mouvements » $I(t)$, mais pas le même nombre de *comparaisons*. Le tri à bulles refait un passage complet ($\Theta(n)$ comparaisons) tant qu'un seul échange subsiste. Exemple : $t = [2, 3, 4, \dots, n, 1]$ a $I(t) = n-1$ inversions ; le tri par insertion coûte $\Theta(n)$, alors que le tri à bulles a besoin de $n-1$ passages (le 1 ne remonte que d'une case par passage) soit $\Theta(n^2)$. **Le tri par insertion est nettement préférable sur un tableau presque trié.**

Exercice 5 — Tri en place, tri stable

Solution. 1. Si les éléments sont de simples nombres, deux éléments égaux sont indiscernables : échanger leurs positions ne se voit pas. La stabilité n'a de sens que pour des *enregistrements* (nom, note, classe...) triés selon une *clé* partielle. *Application.* Pour obtenir une liste d'élèves triée par classe, puis par note à l'intérieur de chaque classe, il suffit de trier d'abord par note, puis par classe **avec un tri stable** : le second tri préserve l'ordre des notes au sein d'une même classe. Avec un tri instable, ce procédé échoue.

2. Soit $t = [2_a, 2_b, 1]$. $i = 0$: $2_b < 2_a$? non ; $1 < 2_a$? oui, donc $\text{imin} = 2$. Échange de $t[0]$ et $t[2]$: $t = [1, 2_b, 2_a]$. $i = 1$: $2_a < 2_b$? non, $\text{imin} = 1$, échange à vide. Résultat : $[1, 2_b, 2_a]$: 2_b précède 2_a alors que 2_a était initialement avant. **Le tri par sélection n'est pas stable.**

3. Dans le tri par insertion, la boucle `while` s'arrête dès que $t[j] \leq \text{cle}$: la clé est donc insérée *juste après* le dernier élément qui lui est inférieur ou égal, donc après tous ses égaux déjà placés — lesquels venaient de positions plus petites. L'ordre relatif des égaux est préservé : le tri est **stable**. Avec `t[j] >= cle`, la boucle continuerait à décaler les éléments égaux et `cle` passerait devant eux : le tri deviendrait instable (tout en restant correct).

4. Oui : l'échange n'a lieu que si $t[j] > t[j+1]$ (inégalité *stricte*), donc deux éléments égaux ne sont jamais permutés. Le tri à bulles est **stable**.

5. Tableau (complété avec les parties II et III) :

Tri	En place	Stable
Sélection	oui	non
Insertion	oui	oui
Bulles	oui	oui
Fusion	non ($\Theta(n)$ auxiliaire)	oui
Rapide (Lomuto)	oui ($\Theta(\log n)$ de pile)	non

Partie II — Le tri par fusion

Exercice 6 — La fonction `fusion` : spécification, invariant, variant

Solution. 1. *Précondition* : A et B sont deux listes triées par ordre croissant (au sens large). *Postcondition* : `fusion(A,B)` renvoie une *nouvelle* liste triée par ordre croissant dont le multienemble des éléments est la réunion de ceux de A et B ; A et B ne sont pas modifiées.

2. *Variant* : $V = (|A| - i) + (|B| - j)$, entier positif tant que la condition de boucle est vraie, et chaque tour incrémente exactement l'un de i ou j , donc V décroît strictement de 1. La boucle termine (en au plus $|A| + |B|$ tours).

3. *Invariant $\mathcal{K}$* :

C est trié; C contient exactement les éléments de $A[0..i-1]$ et $B[0..j-1]$; et $\max C \leq \min(A[i..], B[j..])$ (avec la convention $\max \emptyset = -\infty$).

Initialisation. $i = j = 0$, $C = []$: les trois clauses sont vraies. ✓

Hérédité. Supposons $\mathcal{K}$ vraie et le test de boucle vrai. Posons $v = \min(A[i], B[j])$; comme A et B sont triées, v est le minimum de tous les éléments restants, et par $\mathcal{K}$ il est $\geq \max C$. L'instruction exécutée (`C.append(A[i])` si $A[i] \leq B[j]$, sinon `C.append(B[j])`) ajoute précisément v à la fin de C : C reste trié, la deuxième clause est mise à jour par l'incréméntation correspondante, et la troisième reste vraie puisque v est retiré des restes. ✓

Sortie. La boucle s'arrête quand $i = |A|$ ou $j = |B|$. Supposons $i = |A|$ (l'autre cas est symétrique) : alors $A[i:]$ est vide et $C + B[j:]$ est trié car $\max C \leq \min B[j..]$ et $B[j..]$ est trié. Les éléments sont exactement ceux de A et B . La postcondition est satisfaite. ✓

4. Chaque tour de boucle effectue exactement une comparaison et ajoute exactement un élément à C .

— *Minoration.* La boucle ne s'arrête que lorsqu'une des deux listes est épuisée, ce qui demande au moins $\min(p, q)$ ajouts : $c \geq \min(p, q)$. Borne atteinte si tous les éléments de A sont $\leq$ tous ceux de B (par exemple $A = [1, 2]$, $B = [3, 4, 5]$: $c = 2 = \min(p, q)$).

— *Majoration.* Après $p + q - 1$ ajouts, il ne reste qu'un seul élément, donc l'une des deux listes est nécessairement épuisée et la boucle s'est arrêtée : $c \leq p + q - 1$. Borne atteinte pour des listes « en alternance », par exemple $A = [1, 3, 5]$, $B = [2, 4, 6]$: $c = 5 = p + q - 1$.

Dans tous les cas $c = \Theta(p + q)$, et le coût total (avec les concaténations finales) est $\Theta(p + q)$.

5. La liste C puis la concaténation occupent $\Theta(p + q)$ emplacements : `fusion` n'est **pas** en place.

Exercice 7 — `tri_fusion` : terminaison et correction

Solution. 1. *Fonction de mesure* : $\mu(L) = |L| \in \mathbb{N}$. Si $|L| \geq 2$, alors $m = \lfloor |L|/2 \rfloor$ vérifie $1 \leq m \leq |L| - 1$: en effet $m \geq 1$ car $|L| \geq 2$, et $m \leq |L|/2 < |L|$. Donc $|L[:m]| = m < |L|$ et $|L[m:]| = |L| - m < |L|$ (car $m \geq 1$) : la mesure décroît strictement sur chacun des deux appels. Comme $\mathbb{N}$ est bien fondé, la récursion termine.

Si l'on avait écrit $m = 0$, l'appel `tri_fusion(L[0:])` porterait sur L tout entier : mesure non décroissante, récursion infinie (`RecursionError`). Même problème avec $m = \text{len}(L)$.

2. Récurrence forte sur $n = |L|$. Base : $n \leq 1$, la liste est trivialement triée et renvoyée telle quelle. Hérédité : soit $n \geq 2$ et supposons la propriété vraie pour toute liste de longueur $< n$. Comme $|L[:m]| < n$ et $|L[m:]| < n$, `gauche` et `droite` sont, par hypothèse de récurrence, des permutations triées de $L[:m]$ et $L[m:]$. La précondition de `fusion` est donc satisfaite, et l'exercice 6 garantit que `fusion(gauche, droite)` est une liste triée dont les éléments sont exactement ceux de $L[:m]$ et $L[m:]$, c'est-à-dire ceux de L (car $L = L[:m] + L[m:]$). ✓

3. Non, `tri_fusion` n'est pas en place : chaque appel crée deux tranches puis une liste fusionnée. À un instant donné, la somme des tailles vivantes le long d'une branche vaut $n + \frac{n}{2} + \frac{n}{4} + \dots = \Theta(n)$, et la pile d'appels a une profondeur $\Theta(\log n)$. L'espace auxiliaire est donc $\Theta(n)$ — c'est le prix à payer pour la garantie $\Theta(n \log n)$.

4. Sur $L = [5, 2, 4, 6, 1, 3]$:

<i>découpage</i>	$[5, 2, 4, 6, 1, 3] \rightarrow [5, 2, 4] / [6, 1, 3]$
	$[5, 2, 4] \rightarrow [5] / [2, 4] \rightarrow [2] / [4]$
	$[6, 1, 3] \rightarrow [6] / [1, 3] \rightarrow [1] / [3]$
<i>fusions</i>	$[2] + [4] \rightarrow [2, 4] ; [5] + [2, 4] \rightarrow [2, 4, 5]$
	$[1] + [3] \rightarrow [1, 3] ; [6] + [1, 3] \rightarrow [1, 3, 6]$
	$[2, 4, 5] + [1, 3, 6] \rightarrow [1, 2, 3, 4, 5, 6]$

Exercice 8 — Complexité du tri fusion

Solution. 1. Un appel sur $n \geq 2$ éléments découpe en deux listes de tailles $\lfloor n/2 \rfloor$ et $\lceil n/2 \rceil$, les trie récursivement, puis les fusionne. D'après l'exercice 6, la fusion coûte au plus $n - 1$ comparaisons (et ce pire cas est atteignable). D'où

$$C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + (n - 1), \quad C(1) = C(0) = 0.$$

2. Pour $n = 2^k$ la récurrence devient $C(2^k) = 2C(2^{k-1}) + 2^k - 1$. Posons $D(k) = C(2^k)$: $D(k) = 2D(k-1) + 2^k - 1$, $D(0) = 0$. Homogène : $A2^k$. Particulière pour 2^k : 2 étant racine de l'équation caractéristique, on cherche $\alpha k 2^k$: $\alpha k 2^k = 2\alpha(k-1)2^{k-1} + 2^k = \alpha(k-1)2^k + 2^k$ donne $\alpha = 1$. Particulière pour -1 : $c = 2c - 1$ donne $c = 1$. Donc $D(k) = A2^k + k2^k + 1$ et $D(0) = A + 1 = 0$ donne $A = -1$:

$$D(k) = (k-1)2^k + 1 \quad \implies \quad \boxed{C(n) = n \log_2 n - n + 1}.$$

Vérifications : $n = 2$ ($k = 1$) : $C = 0 \cdot 2 + 1 = 1$ (une comparaison pour fusionner deux singletons) ✓ ; $n = 4$ ($k = 2$) : $C = 1 \cdot 4 + 1 = 5$ ✓.

3. $C(n) = n \log_2 n - n + 1 = \Theta(n \log n)$. Dans le meilleur des cas, chaque fusion coûte au moins $\min(p, q) = \lfloor n/2 \rfloor$ comparaisons, d'où $C_{\min}(n) \geq \frac{n}{2} \log_2 n$: la complexité est encore $\Theta(n \log n)$. **Le tri fusion n'est pas sensible à l'ordre initial** : c'est sa force (garantie) et sa faiblesse (aucun gain sur un tableau presque trié).

4. Montrons $C(n) \leq n \lceil \log_2 n \rceil$ par récurrence forte, en posant $k = \lceil \log_2 n \rceil$. Pour $n = 1$, $C(1) = 0$. Pour $n \geq 2$, on a $\lceil \log_2 \lceil n/2 \rceil \rceil = k - 1$ et $\lceil \log_2 \lfloor n/2 \rfloor \rceil \leq k - 1$, donc par hypothèse de récurrence

$$C(n) \leq \left\lfloor \frac{n}{2} \right\rfloor (k-1) + \left\lceil \frac{n}{2} \right\rceil (k-1) + n - 1 = n(k-1) + n - 1 = nk - 1 \leq n \lceil \log_2 n \rceil. \quad \checkmark$$

5.

n	$n \log_2 n$	$n^2/2$	rapport
10^3	$\approx 9,97 \cdot 10^3$	$5 \cdot 10^5$	≈ 50
10^6	$\approx 2,0 \cdot 10^7$	$5 \cdot 10^{11}$	$\approx 2,5 \cdot 10^4$

À 10^8 opérations par seconde, $n = 10^6$ demande 0,2 s au tri fusion contre environ 1 heure et 20 minutes à un tri quadratique.

Exercice 9 — Stabilité du tri fusion

Solution. 1. La liste **gauche** contient les éléments d'indices initiaux $< m$ et **droite** ceux d'indices $\geq m$. Dans **fusion**, en cas d'égalité le test $A[i] \leq B[j]$ est vrai et c'est l'élément de **A** — donc celui d'indice initial le plus petit — qui est ajouté en premier. Par récurrence forte (chaque moitié est triée de façon stable), l'ordre relatif des éléments de clés égales est préservé : **tri_fusion** est **stable**. Tout tient dans le $\leq$.

2. Avec $A[i] < B[j]$, l'algorithme reste **correct** (le résultat est toujours une permutation triée : en cas d'égalité, l'ordre entre les deux éléments est indifférent pour le tri). Mais il n'est plus **stable**. *Contre-exemple* : $L = [2_a, 2_b]$. Alors $m = 1$, **gauche** = $[2_a]$, **droite** = $[2_b]$. Le test $2_a < 2_b$ est faux (clés égales), donc 2_b est ajouté en premier : le résultat est $[2_b, 2_a]$.

3. Avec $m = 1$, l'algorithme reste correct : les hypothèses de terminaison ($1 \leq m \leq |L| - 1$) et de correction sont toujours vérifiées. Mais la récurrence devient

$$C(n) = C(1) + C(n-1) + \Theta(n) = C(n-1) + \Theta(n) \implies C(n) = \Theta(n^2).$$

On retrouve essentiellement le **tri par insertion** : à chaque étape on insère un élément isolé dans une liste déjà triée. Le découpage *équilibré* est donc ce qui fait tout le gain du tri fusion.

Partie III — Le tri rapide

Exercice 10 — tri_rapide : preuves et cas extrêmes

Solution. 1. *Mesure* : $\mu(L) = |L|$. Les listes **petits** et **grands** sont construites à partir de $L[1:]$: c'est le retrait du pivot qui garantit

$$|\text{petits}| + |\text{grands}| = n - 1 < n,$$

donc chacune est de taille $\leq n - 1 < n$. La mesure décroît strictement : la récursion termine. *Sans* le retrait du pivot (par exemple `[x for x in L if x <= pivot]`), la liste **petits** contiendrait toujours le pivot et pourrait valoir L tout entier : récursion infinie.

2. Récurrence forte sur $n = |L|$. Base : $n \leq 1$, liste triée. Hérédité : par hypothèse de récurrence, **tri_rapide**(**petits**) et **tri_rapide**(**grands**) sont des permutations triées de **petits** et **grands**. Par construction, tout élément de **petits** est $\leq \text{pivot}$ et tout élément de **grands** est $> \text{pivot}$; la concaténation

$$\underbrace{\text{trié, } \leq \text{pivot}}_{\text{tri_rapide}(\text{petits})} \parallel [\text{pivot}] \parallel \underbrace{\text{trié, } > \text{pivot}}_{\text{tri_rapide}(\text{grands})}$$

est donc triée. Enfin **petits**, **[pivot]** et **grands** forment une partition du multiensemble L : le résultat est bien une permutation de L . ✓

3. En comptant une comparaison par élément confronté au pivot, un appel sur n éléments coûte $n - 1$ comparaisons (l'implémentation par deux compréhensions en fait $2(n - 1)$, soit un facteur

constant 2). Avec un pivot médian :

$$C(n) = (n-1) + 2C\left(\frac{n-1}{2}\right).$$

C'est (à un décalage près) la récurrence du tri fusion : par le théorème général ($a = 2$, $b = 2$, $\alpha = 1$, $f(n) = n$, cas 2) on obtient $C(n) = \Theta(n \log n)$.

4. Si L est trié par ordre croissant, $\text{pivot} = L[0] = \min L$, donc aucun élément de $L[1:]$ n'est $<$ pivot au sens strict : **petits** est vide (ou ne contient que les doublons du minimum) et **grands** = $L[1:]$. D'où

$$C(n) = (n-1) + C(n-1), \quad C(0) = 0 \implies C(n) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2).$$

Paradoxe apparent : l'entrée la mieux structurée est la pire pour l'algorithme. Il n'y a pas de contradiction : le tri rapide ne tire sa vitesse que de la *répartition équilibrée* autour du pivot, et prendre systématiquement le premier élément d'une liste triée produit le pire déséquilibre possible. C'est un cas fréquent en pratique (données déjà partiellement ordonnées), d'où l'importance du choix du pivot (exercice 13).

5. Profondeur de pile : $\Theta(\log n)$ dans le meilleur cas, $\Theta(n)$ dans le pire. Pour $n = 10^6$, la version dégénérée provoque un **RecursionError** en Python (limite par défaut ≈ 1000), voire un débordement de pile dans un langage compilé : le problème n'est pas seulement de temps, mais de *mémoire*.

Exercice 11 — Complexité en moyenne du tri rapide

Solution. 1. Si L est une permutation uniforme de n éléments distincts, $L[0]$ est uniformément distribué parmi les n éléments : son rang k (nombre d'éléments qui lui sont strictement inférieurs) suit la loi uniforme sur $\{0, \dots, n-1\}$. Dans ce cas $|\text{petits}| = k$ et $|\text{grands}| = n-1-k$, et ces deux sous-listes sont elles-mêmes des permutations uniformes (propriété classique, admise ici). En conditionnant sur k et par linéarité de l'espérance :

$$C(n) = (n-1) + \frac{1}{n} \sum_{k=0}^{n-1} (C(k) + C(n-1-k)) = (n-1) + \frac{2}{n} \sum_{k=0}^{n-1} C(k),$$

la dernière égalité venant du changement d'indice $k \mapsto n-1-k$ dans la seconde somme.

2. En multipliant par n : $nC(n) = n(n-1) + 2 \sum_{k=0}^{n-1} C(k)$. Au rang $n-1$: $(n-1)C(n-1) = (n-1)(n-2) + 2 \sum_{k=0}^{n-2} C(k)$. Par différence :

$$nC(n) - (n-1)C(n-1) = (n-1)[n - (n-2)] + 2C(n-1) = 2(n-1) + 2C(n-1),$$

soit $\boxed{nC(n) = (n+1)C(n-1) + 2(n-1)}$.

3. En divisant par $n(n+1)$ et en posant $D(n) = C(n)/(n+1)$:

$$D(n) = D(n-1) + \frac{2(n-1)}{n(n+1)}.$$

Décomposition : $\frac{n-1}{n(n+1)} = \frac{A}{n} + \frac{B}{n+1}$ donne $n-1 = A(n+1) + Bn$, d'où $A = -1$ (en $n = 0$) et $B = 2$ (en $n = -1$). Ainsi

$$\frac{2(n-1)}{n(n+1)} = -\frac{2}{n} + \frac{4}{n+1}.$$

4. Comme $D(1) = C(1)/2 = 0$, en sommant de $k = 2$ à n :

$$D(n) = \sum_{k=2}^n \left(-\frac{2}{k} + \frac{4}{k+1} \right) = -2(H_n - 1) + 4\left(H_{n+1} - 1 - \frac{1}{2}\right) = 4H_{n+1} - 2H_n - 4.$$

Avec $H_{n+1} = H_n + \frac{1}{n+1}$: $D(n) = 2H_n + \frac{4}{n+1} - 4$. Enfin

$$C(n) = (n+1)D(n) = 2(n+1)H_n - 4(n+1) + 4 \implies \boxed{C(n) = 2(n+1)H_n - 4n}.$$

Vérifications. $n = 2$: $2 \cdot 3 \cdot \frac{3}{2} - 8 = 9 - 8 = 1$, et deux éléments demandent bien une comparaison.
 $\checkmark n = 3$: $2 \cdot 4 \cdot \frac{11}{6} - 12 = \frac{44}{3} - 12 = \frac{8}{3} \approx 2,67$; directement $C(3) = 2 + \frac{2}{3}(C(0) + C(1) + C(2)) = 2 + \frac{2}{3} = \frac{8}{3}$.
 $\checkmark$

5. $H_n = \ln n + \gamma + o(1)$ donc

$$C(n) \sim 2n \ln n = 2 \ln 2 \cdot n \log_2 n \approx 1,386 n \log_2 n.$$

Le tri fusion est effectuée environ $n \log_2 n$: le tri rapide fait donc en moyenne environ **39** % de comparaisons en plus.

6. Et pourtant il est souvent plus rapide :

- dans sa version en place (exercice 12) il n'alloue aucune mémoire et travaille sur des zones contiguës : bien meilleure *localité de cache* ;
- la boucle interne de la partition est extrêmement simple (une comparaison, un échange conditionnel), donc sa constante multiplicative est petite, alors que la fusion recopie chaque élément dans un tableau auxiliaire.

Une comparaison de complexités ne dit rien des constantes ni du comportement mémoire : c'est la limite de l'analyse asymptotique.

Exercice 12 — Partition en place (schéma de Lomuto)

Solution. 1. Sur $t = [2, 8, 7, 1, 3, 5, 6, 4]$, $g = 0$, $d = 7$, $p = t[7] = 4$, $i = -1$:

j	$t[j]$	$t[j] \leq 4$?	i	tableau après le tour
0	2	oui	0	[2, 8, 7, 1, 3, 5, 6, 4]
1	8	non	0	[2, 8, 7, 1, 3, 5, 6, 4]
2	7	non	0	[2, 8, 7, 1, 3, 5, 6, 4]
3	1	oui	1	[2, 1, 7, 8, 3, 5, 6, 4]
4	3	oui	2	[2, 1, 3, 8, 7, 5, 6, 4]
5	5	non	2	[2, 1, 3, 8, 7, 5, 6, 4]
6	6	non	2	[2, 1, 3, 8, 7, 5, 6, 4]
échange final $t[3] \leftrightarrow t[7]$				[2, 1, 3, 4, 7, 5, 6, 8]

La fonction renvoie $m = 3$.

2. Invariant $\mathcal{P}$: « $t[g..i]$ ne contient que des éléments $\leq p$; $t[i+1..j-1]$ ne contient que des éléments $> p$; $t[d] = p$; et $t[g..d]$ est une permutation du contenu initial ».

Initialisation. $i = g - 1$, $j = g$: les deux tranches sont vides. $\checkmark$

Hérédité. Deux cas.

- $t[j] > p$: rien n'est modifié, j augmente de 1 et la tranche $t[i+1..j-1]$ s'allonge d'un élément $> p$. $\checkmark$
- $t[j] \leq p$: on incrémente i puis on échange $t[i]$ et $t[j]$. Avant l'échange, $t[i]$ (nouveau i , donc l'ancien $i+1$) appartenait à la zone des $> p$; après l'échange, $t[i] = \text{ancien } t[j] \leq p$ rejoint la zone de gauche et l'élément $> p$ est envoyé en position j , qui devient la fin de la zone du milieu après incrémentation de j . $\checkmark$

Sortie. $j = d : t[g..i] \leq p, t[i+1..d-1] > p$ et $t[d] = p$. L'échange final $t[i+1] \leftrightarrow t[d]$ place le pivot en $m = i+1$, envoie en $t[d]$ un élément $> p$ (ou $t[d]$ lui-même si $i+1 = d$). On obtient bien

$$t[g..m-1] \leq p = t[m] < t[m+1..d].$$

3. `partition` est une boucle `for` sur un intervalle fini : elle termine. Pour `tri_rapide_place`, la mesure $\mu(g, d) = d - g + 1 \in \mathbb{N}$ décroît strictement : les appels portent sur $(g, m-1)$ et $(m+1, d)$, de mesures $m-g$ et $d-m$, toutes deux $\leq d-g < d-g+1$ puisque $g \leq m \leq d$. La récursion termine.

4. Exactement $d-g$ comparaisons (une par valeur de j dans `range(g, d)`).

5. Chaque appel n'utilise que $\Theta(1)$ variables auxiliaires, mais la pile d'appels a une profondeur $\Theta(\log n)$ dans le meilleur cas et $\Theta(n)$ dans le pire. L'algorithme est donc « en place » au sens usuel (aucun tableau auxiliaire), mais son espace total est $\Theta(\log n)$ en moyenne — et non $\Theta(1)$ au sens le plus strict. *Remarque pratique* : rappeler récursivement sur la *plus petite* moitié et traiter l'autre par une boucle garantit une pile $O(\log n)$ même dans le pire cas.

6. Non, ce tri n'est pas stable. Sur $t = [2_a, 2_b, 1]$ avec $g = 0, d = 2 : p = t[2] = 1$, aucun $t[j] \leq 1$ pour $j \in \{0, 1\}$, donc i reste -1 et l'échange final $t[0] \leftrightarrow t[2]$ donne $[1, 2_b, 2_a]$. Les deux clés égales ont été permutées. Plus généralement, les échanges de la partition font « sauter » des éléments par-dessus de grandes distances, ce qui détruit l'ordre initial.

Exercice 13 — Choix du pivot et entrées dégénérées

Solution. 1. Si tous les éléments sont égaux à v , alors $p = v$ et le test $t[j] \leq p$ est *toujours* vrai : i est incrémenté à chaque tour et vaut $d-1$ à la sortie, donc $m = i+1 = d$. La partition est maximale déséquilibrée : $[g..d-1]$ de taille $n-1$ et $[d+1..d]$ vide. D'où

$$C(n) = (n-1) + C(n-1) = \frac{n(n-1)}{2} = \Theta(n^2).$$

Même phénomène pour la version par compréhension : $x \leq \text{pivot}$ est vrai pour tous les x , donc `petits=L[1:]` et `grands=[]`. Un tableau constant — cas fréquent en pratique — est donc le pire cas du tri rapide, ce qui est contre-intuitif.

2. Avec un pivot tiré au hasard, le pire cas $\Theta(n^2)$ existe encore comme déroulement possible, mais sa probabilité est infime (il faudrait tomber systématiquement sur un extremum). Ce qui change est fondamental : il n'y a plus d'entrée mauvaise. La complexité devient une variable aléatoire dont l'espérance est $\Theta(n \log n)$ **pour toute entrée**, y compris un tableau déjà trié ; un adversaire qui connaît l'algorithme ne peut plus construire d'entrée piège.

3. Sur un tableau trié, la médiane de $t[g], t[(g+d)/2], t[d]$ est $t[(g+d)/2]$, c'est-à-dire l'élément *médian* de la tranche : la partition est parfaitement équilibrée et la complexité devient $\Theta(n \log n)$. En revanche des entrées quadratiques existent toujours : on peut construire des suites (dites « tueuses de médiane-de-trois ») pour lesquelles ces trois positions sont systématiquement extrêmes. Seule la randomisation supprime la notion d'entrée pire cas.

4. Avec une partition ternaire ($< p$) ($= p$) ($> p$) et récursion sur les seules zones extrêmes :

- tableau constant : une seule partition en $\Theta(n)$, la zone ($= p$) absorbe tout et il n'y a aucun appel récursif : $\boxed{\Theta(n)}$;
- tableau à d valeurs distinctes : chaque niveau élimine au moins une valeur et la profondeur de l'arbre est $O(\log d)$ en moyenne, d'où $\Theta(n \log d)$ — excellent quand $d \ll n$ (notes sur 20, codes postaux, etc.).

Partie IV — Borne inférieure, applications, synthèse

Exercice 14 — La borne $\Omega(n \log n)$: arbre de décision

Solution. 1. Pour $n = 3$ et $t = [a, b, c]$, le tri par insertion compare d'abord a et b , puis insère c (une ou deux comparaisons). L'arbre a $3! = 6$ feuilles et une hauteur de 3 (branche la plus longue : c plus petit que les deux autres). Noter que $\lceil \log_2 6 \rceil = 3$: pour $n = 3$ ce tri est *optimal*.

2. Soient $\sigma \neq \sigma'$ deux permutations des indices. Il existe une entrée dont le tri exige de renvoyer σ , et une autre exigeant σ' . Deux entrées conduisant à la même feuille produisent la même suite de réponses aux comparaisons, donc la même permutation renvoyée. Chaque permutation doit donc être étiquette d'au moins une feuille : l'arbre a au moins $n!$ feuilles.

3. Par récurrence immédiate sur h : un arbre binaire de hauteur 0 a $1 = 2^0$ feuille, et un arbre de hauteur h a au plus deux sous-arbres de hauteur $h - 1$, donc au plus $2 \cdot 2^{h-1} = 2^h$ feuilles. Avec le point **2** : $2^h \geq n!$, soit

$$h \geq \log_2(n!).$$

4. De $n! \geq \left(\frac{n}{e}\right)^n$ on tire

$$\log_2(n!) \geq n \log_2 \frac{n}{e} = n \log_2 n - n \log_2 e = n \log_2 n - 1,4427n.$$

Donc $h = \Omega(n \log n)$: la hauteur de l'arbre étant le nombre de comparaisons dans le pire des cas, **tout tri par comparaisons effectue $\Omega(n \log n)$ comparaisons dans le pire des cas.**

5. Le tri fusion effectue $n \log_2 n - n + 1$ comparaisons au pire, à comparer à la borne $n \log_2 n - 1,44n$: il est **optimal à un terme d'ordre n près**, donc asymptotiquement optimal. Le tri rapide ne l'est pas dans le pire des cas ($\Theta(n^2)$), mais l'est à un facteur constant près ($\approx 1,39$) en moyenne.

6. Parce que le tri par comptage n'effectue *aucune* comparaison entre éléments : il utilise leur valeur comme *indice* d'un tableau de compteurs. Il sort donc du modèle de l'arbre de décision, et la borne ne s'y applique pas : il trie n entiers de $\{0, \dots, k\}$ en $\Theta(n + k)$, ce qui est linéaire lorsque $k = O(n)$. *Il n'y a pas de contradiction : la borne ne vaut que dans le modèle des comparaisons.*

Exercice 15 — Trier pour résoudre

Solution. 1. *Doublons.*

```

1 def doublons(t):
2     u = sorted(t)                # Theta(n log n)
3     for i in range(len(u) - 1):  # Theta(n)
4         if u[i] == u[i+1]:
5             return True
6     return False

```

Correction : deux éléments égaux sont nécessairement *adjacents* dans le tableau trié. Version naïve à deux boucles : $\binom{n}{2} = \Theta(n^2)$ comparaisons. Version par tri : $\Theta(n \log n) + \Theta(n) = \Theta(n \log n)$. *Le tri est ici un prétraitement qui fait tomber la complexité d'un ordre de grandeur.*

2. *Paire de somme s .* On trie ($\Theta(n \log n)$) puis on applique la méthode des deux indices $g = 0$, $d = n - 1$: si $t[g] + t[d] < s$ on incrémente g , si $> s$ on décrémente d , sinon on a trouvé. Cette phase coûte $\Theta(n)$ (chaque tour diminue $d - g$ de 1). Total : $\Theta(n \log n)$, contre $\Theta(n^2)$ pour la version naïve.

3. *Les k plus grands.* (i) k tours de tri par sélection : $\sum_{i=0}^{k-1} (n - i) = \Theta(kn)$. (ii) Tri complet puis lecture des k derniers : $\Theta(n \log n)$. La première est préférable dès que $kn = O(n \log n)$, c'est-à-dire $k = O(\log n)$. Pour k grand (par exemple $k = n/2$), le tri complet l'emporte largement.

4. *Élément majoritaire.* On trie, puis on regarde $u[\lfloor n/2 \rfloor]$ et l'on compte ses occurrences en $\Theta(n)$.
Correction : si x apparaît $c > n/2$ fois dans le tableau trié, ses occurrences occupent un bloc d'indices consécutifs $[s, s + c - 1]$. Si ce bloc ne contenait pas l'indice $\lfloor n/2 \rfloor$, il serait entièrement contenu dans $[0, \lfloor n/2 \rfloor - 1]$ ou dans $[\lfloor n/2 \rfloor + 1, n - 1]$, deux intervalles de longueur $\leq n/2$: contradiction avec $c > n/2$. Donc le majoritaire, s'il existe, est $u[\lfloor n/2 \rfloor]$; le comptage final vérifie qu'il existe bien. Complexité : $\Theta(n \log n)$.

5. *Médiane.* Trier puis lire l'élément central : $\Theta(n \log n)$. (Il existe des algorithmes de sélection en $\Theta(n)$ — « médiane des médianes » — mais ils sortent du cadre de ce TD.)

Exercice 16 — Synthèse et questions de cours

Solution. 1.

Tri	Meilleur	Moyenne	Pire	En place	Stable
Sélection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	oui	non
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	oui	oui
Bulles	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	oui	oui
Fusion	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	non	oui
Rapide (Lomuto)	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	oui*	non

* aucun tableau auxiliaire, mais $\Theta(\log n)$ de pile en moyenne.

2. Vrai ou faux.

- Faux.** Les deux notions sont indépendantes : le tri par sélection est en place et instable, le tri fusion est stable et non en place.
- Faux.** Θ est une propriété *asymptotique* : elle ne dit rien pour n petit ni sur les constantes. En pratique le tri par insertion bat le tri fusion pour $n \lesssim 20$, ce qui justifie les tris hybrides (question 3).
- Faux** tel quel : il faut préciser. Le tri rapide est $\Theta(n \log n)$ *en moyenne* (et dans le meilleur cas) mais $\Theta(n^2)$ *dans le pire des cas*. Une complexité sans quantificateur de cas n'a pas de sens ici.
- Vrai.** La complexité vaut $\Theta(n + I(t))$; si $I(t) = O(n)$ alors $\Theta(n + I(t)) = \Theta(n)$.
- Faux**, deux fois. D'une part la borne porte sur le *pire* des cas : le tri par insertion n'effectue que $n - 1$ comparaisons sur un tableau déjà trié. D'autre part elle ne concerne que les tris *par comparaisons* : le tri par comptage est en $\Theta(n + k)$.

3. Tri hybride.

- La correction est préservée : le cas de base « $|L| \leq S$ » renvoie une permutation triée de L (correction du tri par insertion, exercice 3), ce qui est exactement la précondition requise par **fusion**. La récurrence bien fondée de l'exercice 7 s'applique sans changement, avec un cas de base plus large.
- L'arbre de découpage est arrêté lorsque les blocs ont une taille $\leq S$: il y a donc environ n/S feuilles, chacune triée par insertion en $O(S^2)$, soit $\frac{n}{S} \cdot O(S^2) = O(nS)$ pour l'ensemble des feuilles. Au-dessus, il reste $\log_2(n/S)$ niveaux de fusions coûtant $\Theta(n)$ chacun. Total :

$$\Theta\left(nS + n \log \frac{n}{S}\right).$$

- Cette expression reste $\Theta(n \log n)$ tant que $nS = O(n \log n)$, c'est-à-dire $S = O(\log n)$. Si $S = \Theta(n)$, le terme nS domine et l'on retombe sur $\Theta(n^2)$: le tri hybride dégénère en tri par insertion.

- (d) Parce que la classe Θ masque les constantes. Sur un bloc de 10 à 30 éléments, le tri par insertion ne fait aucune allocation, travaille sur une zone contiguë de mémoire (bonne localité de cache) et a une boucle interne très simple, alors que le tri fusion paie à chaque niveau la création de tranches et la recopie. C'est exactement la stratégie utilisée par les bibliothèques standard (*Timsort* en Python, *introsort* en C++).
-

Fin du corrigé — 16 exercices.